

Computational Communication Science 1 (6 ECTS)

Course File

Saurabh Khanna
Course coordinator and lecturer

Ezgi Dede
Tutorial lecturer

College of Communication
University of Amsterdam

Academic Year 2025–26
Semester 2

Contents

1	Course description and practical information	1
2	Exit qualifications	5
3	Testable objectives	6
4	Planning of testing and teaching	8
5	Literature	9
6	Specific course timetable	10
7	Testing	13
8	Lecturers' team and division of responsibilities	16
9	Calculation of students' study load (in hours)	17
10	Calculation of lecturers' teaching load (in hours)	18

Chapter 1

Course description and practical information

1.1 About the course

Course code	774224002Y
Credits	6 ECTS
Programme	Minor Communication in the Digital Society
Period	Semester 2, academic year 2025–26
Language	English
Format	Seven weeks of lectures and tutorials, followed by a final exam in week 8

Recent developments in digital technologies have fundamentally changed our society and the ways in which we communicate, creating a significant need for basic programming capacities and familiarity with the methods that allow researchers to analyse these trends. This course provides a comprehensive *introduction* to the computational methods and practices used to *study* phenomena in the digital society, including the digital data generated through individuals’ everyday activities.

The course assumes no previous knowledge of computational methods. Students receive a comprehensive introduction to Python for data science, including good practices for data wrangling, basic descriptive and inferential statistics, and data visualisation. Students are also introduced to the basic principles of APIs, open science, and version control with Git. By the end of the course, students will understand key data-science concepts and techniques and will have hands-on experience applying them to a range of tasks.

Programming is taught as a way of understanding how a computer processes instructions, not as a list of commands to memorise. Students practise reading unfamiliar code, reasoning about what it can and cannot do, identifying errors, and writing readable code of their own.

1.2 Learning objectives

Upon completion of the course, students should:

1. have a basic understanding of the use of programming languages for computational communication science and the practices behind open science;

2. have a basic understanding of the most common data types in programming;
3. have a basic understanding of simple control structures, including loops, conditions, and functions;
4. have basic experience with exploring and visualising different data types and executing basic statistical analyses using programming languages;
5. have basic experience with handling errors in programming languages and debugging with the help of online resources;
6. have basic experience with retrieving data using APIs and other data sources; and
7. be able to independently seek, find, and apply solutions for a desired operation in a programming language.

1.3 Entry conditions

The course does not require specialised data-analytics skills or prior knowledge of programming languages. Students are, however, expected to have basic statistical knowledge: they should know what a dataset looks like; understand the ideas behind social-scientific methods; understand correlation and linear regression; interpret plots, tables, and figures; and recognise which statistical analyses are appropriate for particular research questions or hypotheses.

1.4 Attendance requirements

Attendance and active participation in lectures are obligatory. Students are expected to study the literature and take part in live coding. Lectures are not recorded, in order to create a safe learning environment in which students feel free to ask questions and participate actively.

Attendance and active participation in tutorials are a precondition for obtaining the final grade. Students are expected to study the literature, prepare the weekly warm-up exercises, complete the coding challenges, and participate actively in discussions and exercises. Office hours and catch-up sessions are optional and have no attendance requirement.

Non-attendance is permitted only for a valid reason communicated to the relevant lecturer before the lecture or tutorial begins. Students may not miss more than two course meetings. A student who misses a tutorial containing an in-class assignment must contact the lecturers to arrange a resit. Students who miss more than two meetings cannot complete the course.

The general UvA Code of Conduct applies to every course activity.

Study Advisers

The Communication Science Study Advisers are the primary contacts for planning-related questions concerning the Minor. Students should contact them when directed by the lecturers or the course policies. When necessary, they may refer students who are enrolled in another degree programme to that programme's Study Advisers.

1.5 Software requirements

Students need a working Python environment. JupyterLab is the required interface, and installation instructions for JupyterLab Desktop and the supported environment are available on Canvas. The current course repository also documents Anaconda as an installation option because it includes Python, JupyterLab, pandas, and the plotting libraries used later in the course. Special consultation hours are available to troubleshoot installation problems.

1.6 Course structure

The course has fourteen meetings, two per week for seven weeks, followed by the final exam in week 8.

Lecture

In the first meeting each week, the instructor gives short lectures on the week's key topics, demonstrates the concepts through live coding, and hosts a question-and-answer session. Students prepare by studying the assigned literature. Their knowledge is assessed through short questions in selected tutorial weeks.

Tutorial

The second meeting each week is a practicum. Before it, students complete short warm-up coding exercises written for this course. These exercises allow students to assess their progress and prepare questions. At the beginning of selected tutorials, students complete weekly knowledge questions and/or an AI code review. The lecturer then discusses common difficulties and questions from the warm-up exercises. Students spend the remainder of the meeting on the week's coding exercise or in-class programming assignment and can ask for help. Worked solutions are provided for students to check and compare approaches.

1.7 Where to find information

Canvas contains all official course information, announcements, deadlines, and assessment instructions. The code, weekly practice notebooks, tutorial exercises, solutions, and practical Python guidance are available in the 2025–26 course GitHub repository. If information differs between sources, the latest announcement on Canvas is authoritative for deadlines and assessment logistics.

1.8 Communication

Canvas is the main communication channel. The 2025–26 course is taught by Saurabh Khanna and Ezgi Dede. Saurabh is the course coordinator and lecture instructor; Ezgi is the tutorial lecturer.

Use Canvas mail for all course-related communication:

- For questions or logistics concerning the weekly lecture, contact Saurabh.
- For questions or logistics concerning the weekly tutorial, contact Ezgi.
- For assignment questions, contact Ezgi or Saurabh.
- For general concerns, or when unsure whom to contact, contact Saurabh.

For technical problems such as Python questions or error messages, contact the lecturers to arrange an office-hour meeting. Code-related problems cannot normally be resolved by email because the code and its execution need to be examined together.

1.9 Individual support and office hours

Students can obtain support in several ways:

1. During tutorials, students work on weekly coding exercises and can ask the tutorial lecturer for assistance.
2. During office hours, students may ask about assignments, discuss concerns, or inspect code they do not understand. Students with similar questions are encouraged to attend together. Contact the lecturers to arrange an office-hour meeting.
3. Optional online catch-up sessions in weeks 5 and 7 provide further opportunities to revisit earlier material.

1.10 Language

All classes are taught in English. Assignments and the final exam must be completed in English.

1.11 Assessment summary

The final grade consists of weekly knowledge questions (15%), AI code reviews (10%), three in-class programming assignments (30%), and a final coding exam (45%). Chapter 7 specifies the pass thresholds, resit conditions, and deadlines.

Chapter 2

Exit qualifications

The course contributes to the following exit qualifications of the Minor Communication in the Digital Society:

- Skills and abilities
- Problem-solving ability
- Academic thinking

The exit qualifications are elaborated in the following specifications:

- 1.b.3: Regarding specific topics, the graduate is able to understand, position, and critically discuss scientific publications within the field; recognise and acknowledge the contributions of other fields of study; and adopt a position in scientific debate within the field.
- 4.b.2: The graduate is able to assess research within the discipline for its practical usability.
- 4.b.3: The graduate is able to recognise the relevance of other disciplines (interdisciplinarity).
- 4.b.9: The graduate is able to approach the field of study creatively.
- 5.b.5: The graduate is able to revise and expand knowledge relatively independently through study.

Chapter 3

Testable objectives

The table on the following page connects the course's testable learning objectives to the exit qualifications.

Exit qualifications and testable objectives	1.b.3	4.b.2	4.b.3	4.b.9	5.b.5
	Understand, position, and critically discuss scientific publications; recognise contributions from other fields; and adopt a position in scientific debate.	Assess research within the discipline for its practical usability.	Recognise the relevance of other disciplines.	Approach the field of study creatively.	Revise and expand knowledge relatively independently through study.
A	Have a basic understanding of the use of programming languages for computational communication science and the practices behind open science. X	X	X		
B	Have a basic understanding of the most common data types in programming.			X	
C	Have a basic understanding of simple control structures, including loops, conditions, and functions.			X	
D	Have basic experience with exploring and visualising different data types and executing basic statistical analyses using programming languages.			X	
E	Have basic experience with handling errors in programming languages and debugging with the help of online resources.				X
F	Have basic experience with retrieving data using APIs and other data sources.			X	
G	Be able to independently seek, find, and apply solutions for a desired operation in a programming language.			X	X

Chapter 4

Planning of testing and teaching

The course consists of fourteen meetings: one lecture and one tutorial in each of seven teaching weeks. A final coding exam takes place in week 8.

In the first meeting of each teaching week, the instructor introduces the key concepts, demonstrates them through live coding, and hosts a question-and-answer session. Students prepare by studying the assigned literature and other materials. Lectures are not recorded, so that students have a safe environment in which to ask questions and participate actively.

The second meeting is a practicum. Before each tutorial, students complete the weekly warm-up exercises and prepare questions. Tutorials may include knowledge questions, an AI code review, an in-class programming assignment, and a guided coding exercise. The tutorial lecturer discusses common difficulties and supports students while they work on the week's coding challenges. Worked solutions are made available so students can compare approaches.

Optional office hours and catch-up sessions provide additional opportunities to discuss assignments, troubleshoot code, and revisit course material. Code-related problems are normally addressed face-to-face or in an online meeting rather than by email, because they usually require lecturers and students to inspect and run the code together.

The assessment comprises weekly knowledge questions (15%), AI code reviews (10%), three in-class programming assignments (30%), and a final coding exam (45%). Chapter 7 provides the complete grading and resit rules.

Chapter 5

Literature

The schedule in Chapter 6 identifies the mandatory reading for each week. The 2025–26 course has three mandatory articles:

- van Atteveldt and Peng (2018), assigned in week 1;
- van Atteveldt et al. (2019), assigned in week 2; and
- Hilbert et al. (2019), assigned in week 7.

The weekly lecture notebooks, tutorial exercises, practice sets, and worked solutions are core study materials. They are published in the course GitHub repository as the relevant week approaches. Students should work through the practice notebook before consulting its solution.

For broader background and additional examples, students may consult *Computational Analysis of Communication* by van Atteveldt et al. (2022), available online at <https://cssbook.net/>. Any additional mandatory or optional materials will be announced on Canvas.

Chapter 6

Specific course timetable

The weekly sequence below follows the teaching materials in the 2025–26 CCS1 repository. Canvas remains authoritative for room information, deadlines, and any timetable changes.

Before the course: prepare your computer

✓ INSTALL PYTHON AND JUPYTERLAB.

Students must have a working Python and JupyterLab environment before the first class. Follow the installation instructions on Canvas. JupyterLab Desktop and Anaconda are supported routes; Anaconda bundles Python, JupyterLab, pandas, and the plotting libraries used in the course. Consultation hours are available for installation problems.

Week 1: Data types

Lecture

Introduction to Python and JupyterLab notebooks. Students learn how notebooks execute code, including the importance of execution order and kernel state. The lecture introduces `print()` and the core data types: integers, floats, strings, booleans, and lists.

Tutorial

Knowledge questions 1, AI code review 1, and a guided coding exercise on JupyterLab and basic data types.

Mandatory reading

van Atteveldt and Peng (2018).

Week 2: Control flow and for loops

Lecture

The lecture introduces conditional branches using `if`, `elif`, and `else`, followed by loops. While `while` loops are mentioned briefly, the course focuses on `for` loops. The week's

reading places computational work in the context of open and reproducible research.

Tutorial

Knowledge questions 2, AI code review 2, and a guided coding exercise on conditional statements and `for` loops.

Mandatory reading

van Atteveldt et al. (2019).

Week 3: Lists and dictionaries

Lecture

The lecture introduces dictionaries and additional methods for modifying lists. Students revisit `for` loops and combine them with dictionaries. The session also recaps the first two weeks.

Tutorial

In-class programming assignment 1, a guided coding exercise, and feedback on the earlier AI code review. Assignment 1 assesses the material from weeks 1 and 2, including control flow.

From this week onward, an anonymous feedback link is available on Canvas for the rest of the course.

Week 4: Functions

Lecture

The lecture explains why functions are useful, how to define them, how arguments and return values work, and why functions are preferable to repeated code.

Tutorial

In-class programming assignment 2 and a guided coding exercise. Assignment 2 assesses the material from weeks 3 and 4, including lists, dictionaries, and functions.

Week 5: Libraries and functions

Lecture

Students learn how to import libraries and call functions provided by a library. Functions are revisited in this context, and the lecture includes a recap of earlier material.

Tutorial

Knowledge questions 3, AI code review 3, and a guided coding exercise.

An optional online catch-up session provides a question-and-answer opportunity covering the first four weeks.

Week 6: pandas

Lecture

The lecture introduces the **pandas** library, reading data from files, and structuring data in a data frame in preparation for statistical analysis.

Tutorial

In-class programming assignment 3, a guided coding exercise, feedback on the AI code review, and guidance on writing readable code with meaningful comments. Assignment 3 assesses pandas and data analysis.

Week 7: Visualisation and course summary

Lecture

Students learn to create basic visualisations with Python libraries. The lecture then summarises the course and includes a final question-and-answer session.

Tutorial

Knowledge questions 4, a guided coding exercise, feedback on the in-class assignment, and a final question-and-answer session.

An optional online catch-up session covers the whole course.

Mandatory reading

Hilbert et al. (2019).

Week 8: Final exam

The individual coding exam takes place on Monday, 23 March 2026, from 09:00 to 11:00. It follows the format of the in-class assignments but is more extensive: students combine material from across the course to solve coding problems. Chapter 7 sets out the permitted materials and other exam rules.

Chapter 7

Testing

Table 7.1 provides an overview of how the assessment components relate to the learning objectives.

Table 7.1: Test matrix

	Knowledge questions (15%)	AI code review (10%)	In-class assignments (30%)	Final coding exam (45%)
A. Understand programming languages for computational communication science and the practices behind open science.	X	X	X	X
B. Understand the most common data types in programming.	X	X	X	X
C. Understand simple control structures, including loops, conditions, and functions.	X	X	X	X
D. Explore and visualise data and execute basic statistical analyses.	X		X	X
E. Handle programming errors and debug with the help of appropriate resources.		X	X	X
F. Retrieve data using APIs and other data sources.	X		X	X
G. Independently seek, find, and apply solutions for a desired programming operation.		X	X	X

Grading overview

The final grade is composed as follows:

Component	Weight	When
Weekly knowledge questions	15%	Weeks 1, 2, 5, and 7
AI code review	10%	Weeks 1, 2, and 5
In-class programming assignments	30%	Weeks 3, 4, and 6
Final coding exam	45%	Week 8

Weekly knowledge questions (15%)

At the beginning of the tutorials in weeks 1, 2, 5, and 7, students answer six short questions based on the readings and lecture material. Only the best three of the four results count. Students therefore may miss one set without a grading penalty. To pass this component, students must answer at least ten questions correctly across their three counted weeks.

AI code review (10%)

AI tools can generate code quickly, but generated code may contain subtle errors or run without an error while returning the wrong result. The AI code review develops students' ability to read unfamiliar code, trace its execution, identify errors, and correct them.

During the tutorials in weeks 1, 2, and 5, students receive AI-generated code on paper. Without using a computer, interpreter, or external source, each student:

1. identifies and explains the error or errors;
2. writes a corrected version of the code; and
3. photographs the completed work and submits it on Canvas.

The reviews are graded pass/fail. A serious attempt must clearly identify the errors, include corrected code, and be legible. Completing two of the three reviews earns full marks for this component.

In-class programming assignments (30%)

Students complete three individual programming assignments during tutorial meetings. Each assignment is graded on a ten-point scale, and a grade of at least 5.5 is required on *each* assignment.

Assignment	Week	Focus	Share of component
Assignment 1	3	Weeks 1–2 material, including control flow	30%
Assignment 2	4	Weeks 3–4 material, including lists, dictionaries, and functions	30%
Assignment 3	6	pandas and data analysis	40%

Students who cannot attend an in-class assignment because of illness or another valid reason must contact the lecturers as soon as possible to arrange a resit.

Final coding exam (45%)

The final exam follows the format of the in-class assignments but is more extensive and combines material from across the course.

Date	Monday, 23 March 2026
Time	09:00–11:00 (two hours)
Format	Individual coding exam

Students may use only the personal notes added by 17:00 on 22 March to their personal notes folder. Phones, personal laptops, and other electronic devices are not allowed. Students must earn at least 5.5 on the final exam to pass the course. Detailed grading criteria are published on the Canvas exam page.

Use of generative AI

Generative AI is increasingly becoming a standard feature of many coding environments. The course does not prohibit its use during ordinary practice. Instead, students learn to inspect generated code critically and recognise that code can run without raising an error while still producing an incorrect result. Students remain responsible for understanding all code they use. The restrictions stated for individual in-class assignments and the final exam always take precedence; unauthorised tools, sources, and communication may not be used in those assessments.

Resit policy

A student is eligible for a resit only after submitting all assignments on time and meeting the attendance and active-participation requirements.

Component	Resit condition and form	Maximum resit grade
Knowledge questions	Fewer than ten correct answers across the best three weeks: written replacement assignment	6.0
Programming assignments	Grade below 5.5: redo the specific assignment	6.0
Final exam	Grade below 5.5: submit an improved version	6.0

The resit deadline is ten working days after the relevant grade is posted on Canvas. If the improved final exam is still graded below 5.5, the course cannot be completed successfully.

Extensions and exceptions

Students facing exceptional circumstances must contact the lecturers in advance and during working days. In a last-minute emergency, students should contact both the lecturers and the Study Advisers and provide a detailed explanation. Extensions are handled through the Study Advisers and are granted only in extraordinary circumstances.

Fraud and plagiarism

All in-class programming assignments and the final exam are individual assessments. The UvA rules on fraud and plagiarism apply. Work may be checked for plagiarism, and suspected fraud or plagiarism will be reported to the Examinations Board. Students who commit, enable, or assist fraud or plagiarism may be excluded from examinations for up to twelve months.

Language requirement

All assignments and the final exam must be completed in English.

Chapter 8

Lecturers' team and division of responsibilities

Computational Communication Science 1 is taught by Saurabh Khanna and Ezgi Dede. Saurabh is the course coordinator and teaches the lectures. Ezgi teaches the tutorials.

- Questions or logistics concerning the weekly lecture: contact Saurabh.
- Questions or logistics concerning the weekly tutorial: contact Ezgi.
- Questions concerning assignments: contact Ezgi or Saurabh.
- General concerns, or uncertainty about whom to contact: contact Saurabh.

Use Canvas mail for all course-related communication. For technical issues such as Python questions or error messages, contact the lecturers to arrange an office-hour meeting; code-related problems are not handled by email.

Chapter 9

Calculation of students' study load (in hours)

The course carries 6 ECTS, equivalent to 168 hours:

- Attendance at fourteen meetings: 28 hours
- Weekly reading and lecture preparation: 28 hours
- Tutorial preparation and warm-up exercises: 35 hours
- Independent programming practice: 35 hours
- Preparation and follow-up for in-class assignments: 12 hours
- Final exam preparation and examination: 30 hours

Total: 168 hours.

Chapter 10

Calculation of lecturers' teaching load (in hours)

10.1 Course coordinator and lecturer

- Presence: 14 hours (7 meetings of 2 hours)
- Preparation of course materials and notebooks: 26 hours
- Preparation of weekly lectures and knowledge questions: 21 hours
- Preparation of weekly coding challenges and assignments: 14 hours
- Preparation of final exam and datasets: 10 hours
- Administration, communication, appointments, and office hours: 11 hours

Total: 96 hours.

10.2 Tutorial lecturer (per group)

- Presence: 14 hours (7 meetings of 2 hours)
- Preparation of weekly tutorial meetings: 33 hours
- Grading in-class assignments and providing feedback: 9 hours
- Feedback and grading for the final exam: 12 hours

Total: 68 hours.

Literature

- Hilbert, M., Barnett, G., Blumenstock, J., Contractor, N., Diesner, J., Frey, S., Gonzalez-Bailon, S., Lamberso, P., Pan, J., Peng, T.-Q., et al. (2019). Computational communication science: A methodological catalyzer for a maturing discipline. *International Journal of Communication*, 13:3913–3934.
- van Atteveldt, W. and Peng, T.-Q. (2018). When communication meets computation: Opportunities, challenges, and pitfalls in computational communication science. *Communication Methods and Measures*, 12(2–3):81–92.
- van Atteveldt, W., Strycharz, J., Trilling, D., and Welbers, K. (2019). Toward open computational communication science: A practical road map for reusable data and code. *International Journal of Communication*, 13:3935–3954.
- van Atteveldt, W., Trilling, D., and Arcíla Calderón, C. (2022). *Computational Analysis of Communication: A Practical Introduction to the Analysis of Texts, Networks, and Images with Code Examples in Python and R*. John Wiley & Sons, Hoboken, NJ.